

Product Requirement Document (PRD)

Financial & Accounting System Integration (Accurate Core Engine)

Version:	2.0 (Updated for Beverage Distribution)	Author:	Product Management & Engineering Team
Date:	June 28, 2026	Status:	Ready for AI Code Generation

1. Executive Summary & Objective

1.1 Background

As business operations scale, managing financial transactions across e-commerce, point-of-sale (POS), and supply chain platforms through manual reconciliation introduces significant operational overhead and human error. To achieve financial compliance and real-time visibility, the organization requires a standardized, automated bookkeeping engine that either directly integrates with the Accurate Online API or mirrors its core architecture. This will enable automatic journal generation from day-to-day operational events.

1.2 Objectives

- Automation:** Eliminate manual journal entries for standard transactions (Sales, Purchases, Inventory Movements) by triggering automated double-entry bookkeeping via API events.
- Financial Visibility:** Provide standard financial statements (Balance Sheet, Profit & Loss, Cash Flow) instantly.
- Tax Compliance:** Ensure immediate alignment with Indonesian tax regulations, specifically automated dynamic VAT calculation and e-Faktur CSV export compatibility.
- Operational Control:** Optimize beverage distribution logistics by automating multi-unit conversions, tracking product expiration (FIFO), and enforcing customer credit limits.

2. User Personas

Persona / Role	Core Needs	Key Pain Points
Finance Admin Data Inputter & Reconciler	Streamlined transaction capturing, easy invoice matching, and fast bank reconciliation features.	Spends hours matching manual bank statements with separate customer accounts receivable data.
Corporate Accountant Financial Controller & Tax	Strict Chart of Accounts (CoA) enforcement, manual journal adjustments for accruals, and standard tax export formats.	Unbalanced manual adjustments and tedious file format preparation for national tax portals.
Management / Stakeholders Strategic Decision Maker	Real-time business performance dashboards, clear dynamic cash	Delayed visibility into true financial health due to accounting closing taking up to 10 days post-month end.

Persona / Role	Core Needs	Key Pain Points
	runway tracking, and per-brand or project profitability metrics.	

3. Scope & Functional Requirements

3.1 Chart of Accounts (COA) & General Ledger (GL) Engine

- **FR-1.1: Standardized CoA Structure:** The system must enforce a multi-level hierarchical CoA structure compliant with PSAK (Indonesian Accounting Standards), spanning Assets, Liabilities, Equity, Revenue, Cost of Goods Sold (COGS), and Expenses.
- **FR-1.2: Hard Balance Enforcement:** Any transaction trying to write to the ledger must strictly satisfy the equation: $Sum(Debits) == Sum(Credits)$. If unbalanced, the API must reject the payload with a 400 Bad Request error.
- **FR-1.3: Multi-Currency Tracking:** The system must allow transactions in multiple currencies, recording the original transaction value and translating it to IDR (Base Currency) utilizing adjustable daily exchange rates. Realized and unrealized foreign exchange gain/loss must be computed automatically at month-end.

3.2 Sales & Accounts Receivable (AR) API Integration

The system must automatically process sales channels data into accounting records upon confirmation of delivery.

Event Trigger	Accounting Action	Default Journal Generated (Double-Entry)
<code>sales_order.shipped</code>	Generate Sales Invoice, Recognize Revenue and Tax Liability.	Debit: Accounts Receivable (AR) Credit: Sales Revenue Credit: VAT Out (PPN Keluaran)
<code>payment.received</code>	Match Invoice, Record Cash Inflow, Close AR.	Debit: Bank Account / Cash Account Credit: Accounts Receivable (AR)
<code>sales_return.approved</code>	Generate Credit Note, Reverse Revenue, Restock Inventory.	Debit: Sales Return / Revenue Reverse Debit: VAT Out (PPN Keluaran) Credit: Accounts Receivable (AR) (And a simultaneous stock reversal journal for COGS)

- **FR-2.4: Credit Limit & Term of Payment Control:** The system must store customer-specific credit limits and Terms of Payment (TOP, e.g., 14/30 days). If a transaction pushes a customer's total outstanding balance beyond their credit limit, or if they have overdue unpaid invoices, the API must flag a policy violation or block the order creation until approved by management.

4. Inventory Costing & Distribution Engine

- **FR-3.1: Valuation Methods:** Support Moving Average and FIFO valuation rules. Valuation rules must be strictly configured per item category upon initialization and cannot be altered mid-period.

- **FR-3.2: Automated COGS Generation:** Upon inventory reduction due to fulfillment updates, a secondary system journal must be dispatched:
Debit: Cost of Goods Sold (COGS) | **Credit:** Inventory Asset Account.
- **FR-3.3: Multi-Warehouse Tracking:** The system must track inventory across different physical and virtual fulfillment spaces, facilitating internal stock movements without creating a P&L transaction effect.
- **FR-3.4: Multi-UOM & Conversion Rate:** The architecture must support nested units of measure (e.g., 1 Pallet = 40 Cartons, 1 Carton = 24 Bottles). The database must convert transactions seamlessly back to the lowest base unit (e.g., Bottles) for ledger precision.
- **FR-3.5: Batch Tracking & Expiry Dates:** For beverage perishable tracking, the inventory module must log distinct batches containing mandatory batch numbers and expiration dates. Outbound sales operations must deplete stock using a programmatic FIFO sequence relative to batch expiration thresholds.

5. Indonesian Tax Localization

- **FR-4.1: Dynamic VAT Calculation:** The tax engine must compute VAT based on configurable percentage criteria (defaulting to current legal rates such as 11%) mapped to item-level SKU master data (Taxable vs Non-Taxable flags).
- **FR-4.2: DJP e-Faktur CSV Format Support:** The system must compile a batch export function that aggregates all Taxable Sales Invoices within a defined date range into a standardized CSV layout explicitly formatted for importing into the Directorate General of Taxes (DJP) desktop application.
- **FR-4.3: NSFP Lifecycle Management:** Provide a management portal to input a pool of acquired Serial Numbers for Tax Invoices (NSFP) from e-Nofa. The system must auto-assign these numbers chronologically to approved invoices.

6. Technical Architecture & NFRs

6.1 Target Tech Stack Integration

- **Backend Framework: NestJS** (TypeScript). Business logic, journal automation engines, and tax reporting computations must be processed inside strict, testable NestJS services and modules.
- **Database & Auth Core: Supabase** (PostgreSQL + Supabase Auth). Real-time triggers should listen to inventory adjustments for prompt dashboard syncing. PostgreSQL schemas must leverage strict foreign keys and transactional constraints.
- **Frontend User Interface: React.js** (TailwindCSS). Prioritize an elegant SPA layout that focuses on minimal multi-page reloads, rich dashboards, and clean data tables.

6.2 Security & Operational Integrity

- **Data Security:** All financial telemetry and transaction data payloads must be encrypted during transit using TLS 1.3 and at rest via AES-256 standard algorithms. Full Role-Based Access Control (RBAC) powered by Supabase Auth must restrict journal alteration to verified accounting personnel.
- **Strict System Audit Trail:** The system must prevent any physical deletion ("hard delete") of posted financial entries. Any correction must be handled through voiding mechanisms or reversal entries. Every modification must write to an immutable audit ledger logging User ID, timestamp, IP address, old parameters, and new state.
- **API Idempotency:** To handle network retries safely from POS or operational platforms, the ledger engine must enforce unique `idempotency_key` handling on mutating endpoints to avoid duplicate journal booking.

- **Query Performance:** Financial report generation queries spanning up to 100,000 line items must render in less than 5 seconds.

7. API Endpoint Specifications

7.1 POST /api/v1/sales-invoice

Sample Payload structure required from external operational modules:

```
{
  "idempotency_key": "c39a8c6a-be3d-4c31-9f93-cb7e1897c83f",
  "invoice_number": "INV/20260628/0001",
  "transaction_date": "2026-06-28",
  "customer_id": "CUST-BRW-099",
  "term_of_payment_days": 14,
  "currency": "IDR",
  "exchange_rate": 1.0,
  "items": [
    {
      "sku": "BRW-WLNS-01",
      "quantity": 10,
      "uom": "carton",
      "conversion_to_base": 24,
      "unit_price": 250000.00,
      "taxable": true,
      "batch_number": "BATCH-2026-A2",
      "expiry_date": "2027-06-28"
    }
  ],
  "discount_total": 0.00,
  "warehouse_id": "WH-KEBAGUSAN-01"
}
```

7.2 Expected API JSON Response

```
{
  "status": "success",
  "invoice_id": "TXN-AR-99201",
  "journal_reference": "JV/20260628/8810",
  "total_amount": 2775000.00,
  "vat_amount": 275000.00,
  "posted_at": "2026-06-28T14:28:41Z"
}
```